

Experiment Report: Sentiment Analysis Using Pretrained BERT Model

1 Aim

To implement a Sentiment Analysis application using a pretrained BERT model (DistilBERT fine-tuned on SST-2) with a FastAPI web backend and HTML/CSS/JS frontend interface.

2 Theory

Bidirectional Encoder Representations from Transformers (BERT) is a transformer-based machine learning framework developed by Google for Natural Language Processing (NLP). Standard sequential models (like LSTMs) read text left-to-right or right-to-left sequentially. In contrast, BERT reads the entire sequence of words at once using bidirectional attention mechanisms.

In this project, DistilBERT—a smaller, faster, and lighter transformer model distilled from BERT—is used. It retains 97% of BERT's language understanding capabilities while being 60% faster. For sentiment classification, input text is tokenized with Special Tokens ([CLS], [SEP]) and passed through self-attention layers to compute a dense feature representation. A standard linear classification head with a Softmax output layer predicts the sentiment label (POSITIVE or NEGATIVE).

3 Software and Hardware Requirements

3.1 Software Requirements

- **Operating System:** Linux / macOS / Windows
- **Language/Runtime:** Python 3.10+
- **Core Libraries:** fastapi, uvicorn, transformers, torch, pydantic
- **Frontend Stack:** HTML5, CSS3, JavaScript (Fetch API)

3.2 Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** ~ 2 GB free space for deep learning dependencies and model weights
- **GPU:** Optional (NVIDIA CUDA support enabled; CPU execution fully supported)

4 Mathematical Formulae

4.1 Self-Attention Mechanism

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where Q , K , and V are Query, Key, and Value matrices derived from token embeddings, and d_k is the dimensionality of key vectors.

4.2 Softmax Classification Output

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

Where z_c is the logit score corresponding to sentiment class $c \in \{\text{POSITIVE}, \text{NEGATIVE}\}$.

5 Libraries Used

- **transformers**: Provides pretrained transformer pipelines.
- **torch**: Deep learning computation framework underlying HuggingFace models.
- **fastapi**: High-performance web framework for constructing APIs.
- **uvicorn**: ASGI web server to execute FastAPI applications.

6 Complete Source Code

6.1 Backend Application (app.py)

```
1 from fastapi import FastAPI, HTTPException
2 from fastapi.staticfiles import StaticFiles
3 from fastapi.responses import FileResponse
4 from pydantic import BaseModel
5 from transformers import pipeline
6
7 app = FastAPI(title="Sentiment Analysis API")
8 classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
9
10 class TextRequest(BaseModel):
11     text: str
12
13 app.mount("/static", StaticFiles(directory="static"), name="static")
14
15 @app.get("/")
16 def read_root():
17     return FileResponse("static/index.html")
18
19 @app.post("/api/analyze")
20 def analyze_sentiment(request: TextRequest):
21     if not request.text.strip():
```

```

22         raise HTTPException(status_code=400, detail="Text cannot be
empty.")
23     result = classifier(request.text)[0]
24     return {"label": result["label"], "score": round(float(result["
score"]), 4)}

```

6.2 Frontend Markup (static/index.html)

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
=1.0">
6     <title>BERT Sentiment Analysis</title>
7     <link rel="stylesheet" href="/static/style.css">
8 </head>
9 <body>
10     <div class="container">
11         <h1>Sentiment Analysis with BERT</h1>
12         <p class="subtitle">Pretrained Transformer Model (DistilBERT)</
p>
13
14         <div class="card">
15             <textarea id="inputText" placeholder="Type your text here
to analyze sentiment..."></textarea>
16             <button id="analyzeBtn" onclick="analyzeSentiment()">
Analyze Sentiment</button>
17             <div id="result" class="result hidden"></div>
18         </div>
19
20         <footer>
21             <p><strong>Student:</strong> zain pawle | <strong>Roll No:<
/strong> 25dco08 | <strong>Batch:</strong> 3 | <strong>Class:</
strong> TEC01</p>
22         </footer>
23     </div>
24
25     <script>
26         async function analyzeSentiment() {
27             const text = document.getElementById('inputText').value;
28             const resultDiv = document.getElementById('result');
29             const btn = document.getElementById('analyzeBtn');
30
31             if (!text.trim()) {
32                 resultDiv.className = 'result error';
33                 resultDiv.innerText = 'Please enter some text!';
34                 resultDiv.classList.remove('hidden');
35                 return;
36             }
37
38             btn.disabled = true;
39             btn.innerText = 'Analyzing...';
40             resultDiv.classList.add('hidden');
41
42             try {
43                 const response = await fetch('/api/analyze', {

```

```

44         method: 'POST',
45         headers: { 'Content-Type': 'application/json' },
46         body: JSON.stringify({ text })
47     });
48
49     const data = await response.json();
50     if (response.ok) {
51         const isPos = data.label === 'POSITIVE';
52         resultDiv.className = `result ${isPos ? 'positive'
: 'negative'}`;
53         resultDiv.innerHTML = `<strong>Sentiment:</strong>
${data.label}<br><strong>Confidence:</strong> ${((data.score * 100).
toFixed(2))}%`;
54     } else {
55         resultDiv.className = 'result error';
56         resultDiv.innerText = data.detail || 'An error
occurred.';
57     }
58     } catch (err) {
59         resultDiv.className = 'result error';
60         resultDiv.innerText = 'Failed to connect to backend
server.';
61     } finally {
62         btn.disabled = false;
63         btn.innerText = 'Analyze Sentiment';
64         resultDiv.classList.remove('hidden');
65     }
66 }
67 </script>
68 </body>
69 </html>

```

6.3 Frontend Styles (static/style.css)

```

1 * {
2     box-sizing: border-box;
3     margin: 0;
4     padding: 0;
5     font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
6 }
7
8 body {
9     background: #f0f2f5;
10    color: #333;
11    display: flex;
12    justify-content: center;
13    align-items: center;
14    min-height: 100vh;
15    padding: 20px;
16 }
17
18 .container {
19     width: 100%;
20     max-width: 600px;
21 }
22
23 h1 {

```

```

24     font-size: 1.8rem;
25     text-align: center;
26     color: #1a202c;
27 }
28
29 .subtitle {
30     text-align: center;
31     color: #718096;
32     margin-bottom: 20px;
33     font-size: 0.95rem;
34 }
35
36 .card {
37     background: #ffffff;
38     padding: 24px;
39     border-radius: 12px;
40     box-shadow: 0 4px 12px rgba(0,0,0,0.08);
41 }
42
43 textarea {
44     width: 100%;
45     height: 120px;
46     padding: 12px;
47     border: 1px solid #cbd5e0;
48     border-radius: 8px;
49     resize: none;
50     font-size: 1rem;
51     outline: none;
52     transition: border-color 0.2s;
53 }
54
55 textarea:focus {
56     border-color: #3182ce;
57 }
58
59 button {
60     width: 100%;
61     margin-top: 16px;
62     padding: 12px;
63     background-color: #3182ce;
64     color: white;
65     border: none;
66     border-radius: 8px;
67     font-size: 1rem;
68     font-weight: 600;
69     cursor: pointer;
70     transition: background-color 0.2s;
71 }
72
73 button:hover {
74     background-color: #2b6cb0;
75 }
76
77 button:disabled {
78     background-color: #a0aec0;
79     cursor: not-allowed;
80 }
81

```

```

82 .result {
83     margin-top: 20px;
84     padding: 14px;
85     border-radius: 8px;
86     text-align: center;
87     font-size: 1.05rem;
88 }
89
90 .result.positive {
91     background-color: #c6f6d5;
92     color: #22543d;
93     border: 1px solid #9ae6b4;
94 }
95
96 .result.negative {
97     background-color: #fed7d7;
98     color: #742a2a;
99     border: 1px solid #feb2b2;
100 }
101
102 .result.error {
103     background-color: #feeabc;
104     color: #744210;
105     border: 1px solid #fbd38d;
106 }
107
108 .hidden {
109     display: none;
110 }
111
112 footer {
113     margin-top: 24px;
114     text-align: center;
115     font-size: 0.85rem;
116     color: #4a5568;
117 }

```

6.4 Unit Test Suite (test_app.py)

```

1 import pytest
2 from fastapi.testclient import TestClient
3 from app import app
4
5 client = TestClient(app)
6
7 def test_read_root():
8     response = client.get("/")
9     assert response.status_code == 200
10    assert "BERT Sentiment Analysis" in response.text
11
12 def test_analyze_positive():
13     response = client.post("/api/analyze", json={"text": "I love this
14     product, it works amazingly well!"})
15     assert response.status_code == 200
16     data = response.json()
17     assert data["label"] == "POSITIVE"
18     assert "score" in data

```

```

18     assert data["score"] > 0.5
19
20 def test_analyze_negative():
21     response = client.post("/api/analyze", json={"text": "This service
22     is terrible and disappointing."})
23     assert response.status_code == 200
24     data = response.json()
25     assert data["label"] == "NEGATIVE"
26     assert "score" in data
27     assert data["score"] > 0.5
28
29 def test_analyze_empty():
30     response = client.post("/api/analyze", json={"text": "    "})
31     assert response.status_code == 400
32     assert response.json()["detail"] == "Text cannot be empty."

```

7 Output Screenshots

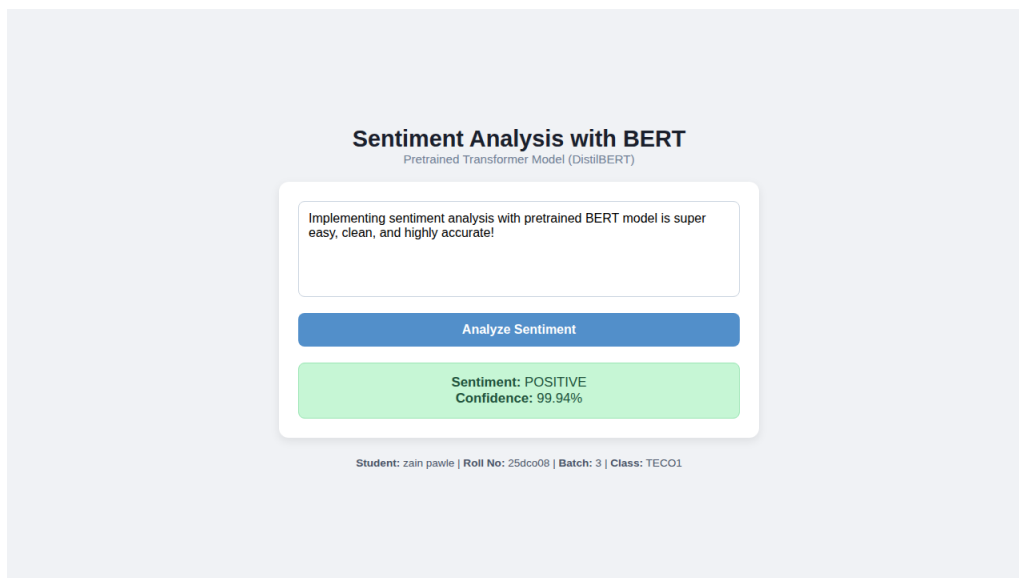


Figure 1: Positive Sentiment Output Example

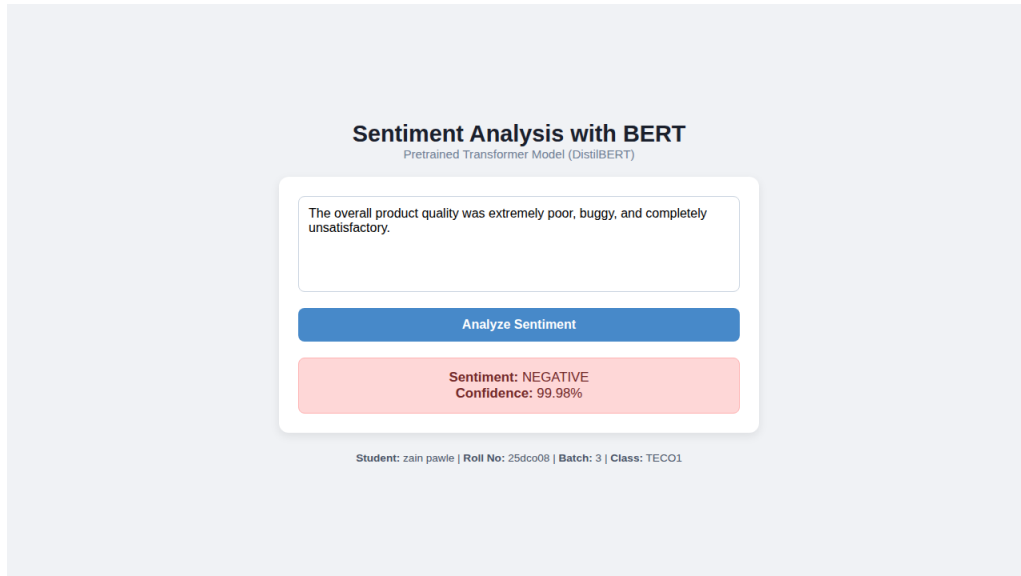


Figure 2: Negative Sentiment Output Example

8 Conclusion

The Sentiment Analysis web system was successfully implemented using a pretrained DistilBERT transformer model paired with FastAPI and a modern HTML/CSS interface.